



UNIVERSIDAD DE MONTERREY

Práctica 12

Microprocesadores

M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo García Mat: 612348

Nombre: Ricardo Garza Benítez Mat: 645972

"Damos nuestra palabra de que hemos hecho esta actividad con integridad académica"

Monterrey, N. L., a 23 de junio del 2026

Introducción

La interacción entre un microcontrolador y el mundo exterior es uno de los retos más recurrentes en el desarrollo de sistemas embebidos. Para que un dispositivo sea verdaderamente útil, necesita al menos dos cosas: un medio de entrada que le permita recibir datos del usuario y un medio de salida que le permita mostrar información de forma comprensible. En esta práctica se combinan ambos elementos mediante la integración de un teclado matricial 4×4 y una pantalla LCD de 16×2 con el microcontrolador PIC16F887, formando la base de una calculadora de cuatro operaciones.

El teclado matricial organiza dieciséis teclas en una matriz de cuatro filas y cuatro columnas, reduciendo a ocho el número de líneas de E/S necesarias para su lectura. Su operación se basa en el escaneo secuencial de filas y la detección de columnas activas, lo que permite identificar qué tecla fue presionada sin ambigüedad. Por su parte, la LCD HD44780 proporciona una interfaz de texto de bajo costo que puede operar en modo de cuatro bits, optimizando el uso de los pines del microcontrolador.

La práctica se dividió en dos actividades progresivas: la primera consistió en hacer funcionar el teclado y mostrar los dígitos pulsados directamente en la LCD; la segunda extendió esa base para implementar una calculadora funcional capaz de realizar sumas, restas, multiplicaciones y divisiones con dos operandos, mostrando el resultado al presionar la tecla igual. A lo largo de ambas actividades se trabajó con el compilador MPLAB XC8 sobre el PIC16F887 operando con un cristal de cuarzo de 16 MHz.

Marco Teórico

Teclado matricial 4×4

Un teclado matricial organiza sus contactos en una cuadrícula de filas y columnas de modo que N filas y M columnas dan acceso a N×M teclas usando solo N+M líneas de E/S. En la configuración de 4×4 habitual, el microcontrolador activa una fila a la vez llevándola a nivel lógico bajo mientras mantiene las demás en alto; a continuación lee las cuatro columnas y, si alguna cae a nivel bajo, la intersección fila-columna identifica de forma inequívoca la tecla pulsada. Este proceso se repite cíclicamente con una frecuencia suficiente para capturar cualquier pulsación humana, técnica conocida como escaneo de teclado por multiplexado de filas. Los puertos de entrada deben configurarse con resistencias de pull-up, ya sea externas o mediante los registros internos del PIC, para garantizar lecturas estables cuando ninguna tecla está presionada [1].

Pantalla LCD HD44780 en modo de 4 bits

El controlador HD44780, presente en la gran mayoría de módulos LCD alfanuméricos de 16×2, acepta dos modos de transferencia de datos: el modo de 8 bits, que envía un byte completo en un solo ciclo de habilitación, y el modo de 4 bits, que divide el byte en dos nibbles consecutivos. En aplicaciones con microcontroladores de pines limitados se prefiere el modo de 4 bits porque libera cuatro líneas de E/S sin penalización significativa de velocidad, dado que la tasa de actualización visual requerida es bastante menor a la capacidad de comunicación del dispositivo. La secuencia de inicialización requiere enviar comandos específicos para fijar el modo de operación, el número de líneas, el tipo de

cursor y el estado de la pantalla, respetando los tiempos mínimos de espera establecidos en la hoja de datos para que el controlador interno quede correctamente configurado [2].

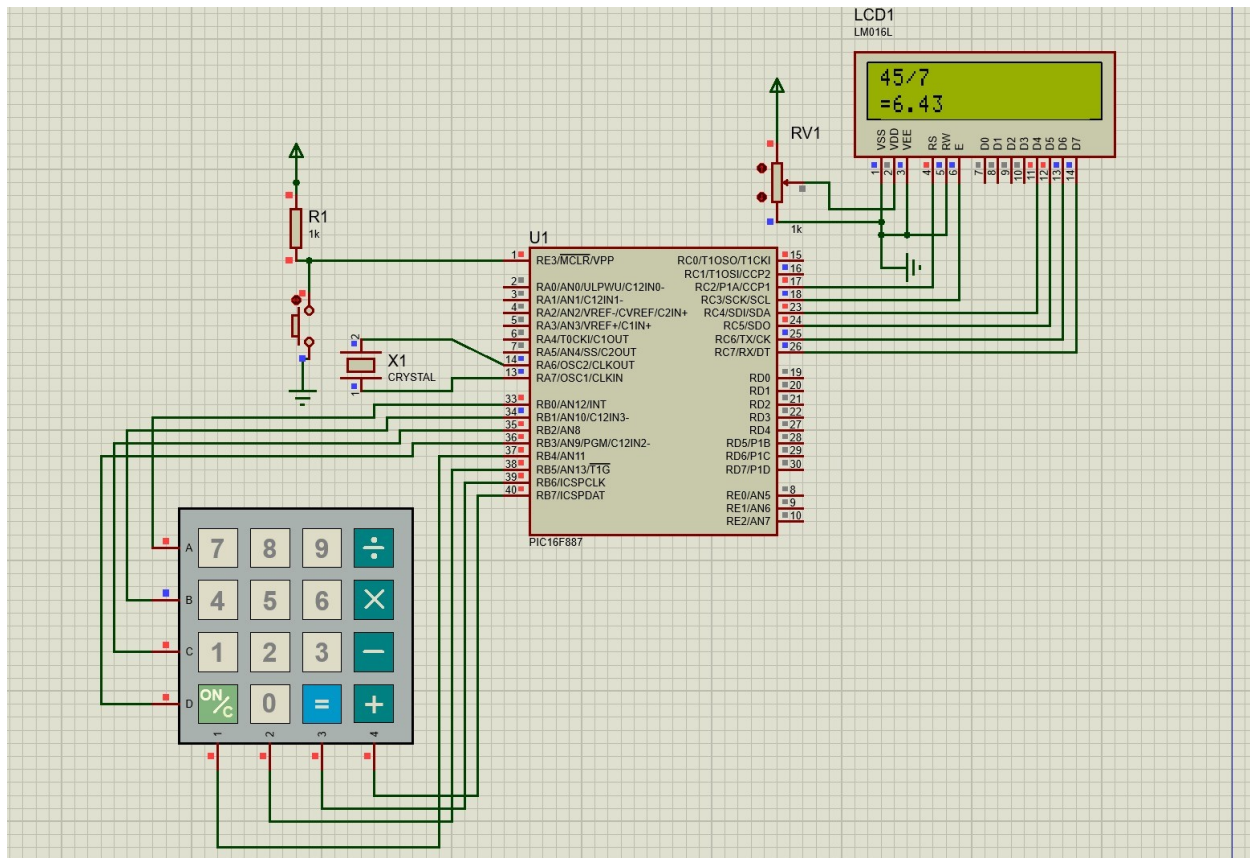
Aritmética de punto flotante en XC8

El compilador MPLAB XC8 soporta el tipo double de 32 bits en su versión gratuita, lo que equivale a un número en formato IEEE 754 de precisión simple. Esta representación almacena el valor en tres campos: un bit de signo, ocho bits de exponente con sesgo 127 y veintitrés bits de mantisa normalizada, lo que proporciona aproximadamente seis a siete dígitos decimales significativos. Para presentar resultados legibles en la LCD se utiliza una función de conversión propia que separa la parte entera de la decimal mediante multiplicación y truncado entero, evitando así las rutinas de printf que incrementan considerablemente el tamaño del binario compilado en arquitecturas de ocho bits [1].

Configuración de puertos y registros TRIS/ANSEL en el PIC16F887

El PIC16F887 dispone de cinco puertos de E/S de propósito general (PORTA a PORTE) cuyos pines pueden funcionar como digitales o analógicos según el contenido de los registros ANSEL y ANSELH. Por defecto, tras un reset, todos los pines de los puertos A y E se inicializan como entradas analógicas; si no se deshabilita este modo explícitamente escribiendo cero en ANSEL y ANSELH, las lecturas digitales producirán valores indefinidos sin importar el estado físico del pin. La dirección de cada pin se controla con los registros TRIS correspondientes: un bit en uno configura el pin como entrada y un bit en cero lo configura como salida. En esta práctica PORTC se configuró completamente como salida para manejar la LCD, mientras que PORTB se usó como interfaz bidireccional para el escaneo del teclado matricial [1].

Simulación



Resultados

Actividad 1 — Lectura del teclado matricial y despliegue en LCD

Durante la primera actividad se logró que cada tecla pulsada en el teclado matricial 4×4 apareciera de forma inmediata en la primera línea de la LCD. El sistema escaneaba las filas del teclado de manera secuencial y, al detectar una columna activa, determinaba el carácter correspondiente de la tabla de mapeo y lo enviaba a la pantalla mediante la función LCD_putc. Se observó que los caracteres se imprimían de izquierda a derecha y, al alcanzar la posición 16, la línea se reiniciaba automáticamente para evitar desbordamiento.

El principal inconveniente que se encontró en esta etapa fue que algunas filas del teclado estaban invertidas respecto al mapeo definido en el archivo Keypad.c. Esto provocaba que al presionar, por ejemplo, la tecla '7', la LCD mostrara un carácter incorrecto. El problema se corrigió reordenando el arreglo de caracteres dentro del driver del teclado para que coincidiera con la distribución física del módulo utilizado. Una vez aplicada la corrección, la lectura fue completamente consistente con las teclas físicas.

Se verificaron todas las dieciséis teclas del teclado, incluyendo los operadores aritméticos y la tecla de borrado, confirmando que ninguna producía un carácter erróneo ni quedaba sin respuesta. El tiempo de respuesta percibido fue prácticamente instantáneo gracias al ciclo de escaneo continuo implementado en el bucle principal.

Actividad 2 — Calculadora funcional

En la segunda actividad se extendió la funcionalidad del sistema para implementar una calculadora de cuatro operaciones. Al ingresar el primer operando y presionar un operador aritmético (+, -, ×, ÷), el PIC almacenaba el valor acumulado y el tipo de operación en variables internas. Al ingresar el segundo operando y presionar '=', el microcontrolador ejecutaba la operación correspondiente mediante la función aplicar y mostraba el resultado en la segunda línea de la LCD precedido del símbolo igual, con dos decimales de precisión. En la simulación de Proteus se comprobó, por ejemplo, que la operación $45 \div 7$ producía correctamente el resultado 6.43 en pantalla.

Se probó también el manejo de errores para la división entre cero: al intentar dividir cualquier valor entre 0, la segunda línea desplegaba el mensaje 'Error: /0' en lugar de un resultado numérico, protegiendo al sistema de un comportamiento indefinido. La tecla 'C' funcionó correctamente como reset completo, limpiando la pantalla y reiniciando todas las variables internas a sus valores iniciales.

Se observó además que la calculadora admitía operaciones encadenadas: tras obtener un resultado con '=', era posible presionar directamente un nuevo operador para continuar operando sobre el resultado previo sin necesidad de limpiar manualmente la pantalla. Este comportamiento fue intencionado en el diseño del código y resultó consistente en todas las pruebas realizadas.

Conclusiones Individuales

Ricardo Garza Benítez

Esta práctica fue de las más completas que hemos tenido porque realmente juntó varias cosas que habíamos trabajado por separado: la configuración de puertos, la LCD, el manejo de variables y la lógica de una aplicación real. Lo que más me costó en el armado físico fue justamente el problema de las filas invertidas del teclado. Al principio pensé que era un error de cableado, así que revisé todas las conexiones físicas en la protoboard varias veces antes de caer en cuenta de que el arreglo de caracteres en el driver simplemente no coincidía con la distribución del módulo que teníamos. Eso me dejó una lección importante: cuando algo falla, vale la pena revisar también el software de bajo nivel y no solo el hardware físico.

La parte de la calculadora me pareció técnicamente interesante porque implicó gestionar estado dentro del microcontrolador: qué operación está pendiente, qué número se está construyendo tecla a tecla, si ya se evaluó una operación o no. Ese tipo de manejo de estados es algo que normalmente uno da por sentado en una calculadora de escritorio, pero implementarlo desde cero en un PIC de ocho bits, con aritmética flotante manual y sin sistema operativo, hace que se valore mucho más el trabajo que hay detrás de cualquier dispositivo embebido cotidiano.

Eugenio Romo García

En esta práctica pude entender de forma más clara cómo un microcontrolador puede actuar como núcleo de una aplicación completa al integrar periféricos de entrada y salida. El teclado matricial fue un ejemplo muy concreto de cómo la multiplexación permite reducir el consumo de pines sin perder funcionalidad, y la LCD mostró que con solo seis

líneas de datos es posible presentar información de texto de manera legible para el usuario.

Lo que más me aportó conceptualmente fue ver cómo la lógica de una calculadora se traduce en variables de estado dentro del programa. La distinción entre el número que se está ingresando, el operando acumulado y el operador pendiente refleja una forma de pensar en máquinas de estados que aplica a prácticamente cualquier sistema interactivo. Creo que esta práctica es un buen punto de cierre para el semestre porque integra los periféricos más relevantes del PIC en una sola aplicación funcional.

Referencias

- [1] Microchip Technology. (2009). PIC16F887 Data Sheet. <https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>
- [2] Hitachi Semiconductor. (1998). HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver. <https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>
- [3] Merino Pérez, L. A. (2012). Microcontroladores PIC: Diseño práctico de aplicaciones. McGraw-Hill.